

K L Deemed to be University

Department of Computer Science and Engineering | 2026-2027, Odd Semester
Database Systems Engineering and Distributed Backend Development (25CS1302E)

Skillng Session 6: MongoDB Aggregation and Design Patterns

Name	Habeeb	Roll No.	2520090054
Course	25CS1302E	CO Mapping	CO2
Session	Skillng 6	Date	

Aim

To build aggregation pipelines with \$match, \$group, \$project, \$sort, \$limit, \$lookup (the equivalent of a SQL JOIN) and \$unwind, and to implement the Polymorphic, Bucket, Attribute, Computed and Outlier schema design patterns for Task Hub.

Program

aggregation_patterns.py

```
"""
Skillng 6: MongoDB aggregation pipeline and schema design patterns - Task Hub.

Run: pip install pymongo
     python aggregation_patterns.py
"""
import os
from datetime import datetime
from pprint import pprint

from pymongo import ASCENDING, MongoClient

client = MongoClient(os.getenv("MONGO_URI", "mongodb://localhost:27017"))
db = client["taskhub_db"]
for name in ("users", "tasks", "activities", "time_buckets", "products", "projects", "watchers_overflow"):
    db.drop_collection(name)

def show(title, docs):
    print(f"\n--- {title} ---")
    for doc in docs:
        pprint(doc, sort_dicts=False, width=110)

db.users.insert_many([
    {"_id": 1, "name": "Habeeb", "team": "Backend"},
    {"_id": 2, "name": "Ananya Rao", "team": "Frontend"},
    {"_id": 3, "name": "Rahul Verma", "team": "Backend"},
    {"_id": 4, "name": "Sneha Reddy", "team": "Frontend"},
])
db.tasks.insert_many([
    {"_id": 101, "title": "Design ER diagram", "project": "Backend", "status": "DONE",
     "assignee_id": 1, "hours": 6, "tags": ["db", "design"]},
    {"_id": 102, "title": "Create PostgreSQL schema", "project": "Backend", "status": "DONE",
     "assignee_id": 3, "hours": 8, "tags": ["db", "sql"]},
    {"_id": 103, "title": "Write FastAPI CRUD", "project": "Backend", "status": "IN_PROGRESS",
     "assignee_id": 1, "hours": 12, "tags": ["api", "python"]},
    {"_id": 104, "title": "Add JWT authentication", "project": "Backend", "status": "TODO",
     "assignee_id": 3, "hours": 2, "tags": ["api", "security"]},
    {"_id": 105, "title": "Build dashboard page", "project": "Frontend", "status": "IN_PROGRESS",
     "assignee_id": 2, "hours": 9, "tags": ["ui", "api"]},
    {"_id": 106, "title": "Replace mock data", "project": "Frontend", "status": "TODO",
     "assignee_id": 4, "hours": 1, "tags": ["ui", "api"]},
])

# =====
# PART 1: AGGREGATION PIPELINE
# =====
# $match -> $group -> $sort : hours per project (only tasks that are not TODO)
show("1. $match + $group + $sort: hours per project (excluding TODO)", db.tasks.aggregate([
    {"$match": {"status": {"$ne": "TODO"}}},
    {"$group": {"_id": "$project", "tasks": {"$sum": 1}, "total_hours": {"$sum": "$hours"},
               "avg_hours": {"$avg": "$hours"}}},
])
```

```

    {"$sort": {"total_hours": -1}},
  ))

# $project : reshape documents and compute new fields
show("2. $project: computed fields", db.tasks.aggregate([
  {"$match": {"project": "Backend"}},
  {"$project": {"_id": 0, "title": 1, "hours": 1,
    "is_done": {"$eq": [{"$status", "DONE"}]},
    "tag_count": {"$size": "$tags"},
    "effort": {"$cond": [{"$gte": [{"$hours", 8}], "large", "small"]}}},
  ]))

# $unwind + $group : how often each tag is used, top 3 with $limit
show("3. $unwind + $group + $sort + $limit: top 3 tags", db.tasks.aggregate([
  {"$unwind": "$tags"},
  {"$group": {"_id": "$tags", "count": {"$sum": 1}}},
  {"$sort": {"count": -1, "_id": 1}},
  {"$limit": 3},
  ]))

# $lookup : equivalent of a SQL LEFT OUTER JOIN between tasks and users
show("4. $lookup (JOIN tasks -> users) + $unwind + $project", db.tasks.aggregate([
  {"$match": {"status": "IN_PROGRESS"}},
  {"$lookup": {"from": "users", "localField": "assignee_id",
    "foreignField": "_id", "as": "assignee"}},
  {"$unwind": "$assignee"},
  {"$project": {"_id": 0, "title": 1, "assignee": "$assignee.name", "team": "$assignee.team"}},
  ]))

# Full report: hours per team (join, group, sort)
show("5. Team workload report ($lookup + $group + $sort)", db.tasks.aggregate([
  {"$lookup": {"from": "users", "localField": "assignee_id", "foreignField": "_id", "as": "u"}},
  {"$unwind": "$u"},
  {"$group": {"_id": "$u.team", "members": {"$addToSet": "$u.name"},
    "open_tasks": {"$sum": {"$cond": [{"$ne": [{"$status", "DONE"}]}, 1, 0]}},
    "hours": {"$sum": "$hours"}},
  {"$sort": {"hours": -1}},
  {"$project": {"_id": 0, "team": "$_id", "hours": 1, "open_tasks": 1, "members": 1}},
  ]))

# =====
# PART 2: SCHEMA DESIGN PATTERNS
# =====
# --- Polymorphic: documents of different shapes in ONE collection, told apart by "type"
db.activities.insert_many([
  {"type": "comment", "task_id": 103, "user_id": 1, "text": "Started GET /tasks"},
  {"type": "status_change", "task_id": 103, "user_id": 1, "from": "TODO", "to": "IN_PROGRESS"},
  {"type": "file_upload", "task_id": 101, "user_id": 1, "file_name": "er.png", "size_kb": 292},
])
show("6. POLYMORPHIC pattern: activity feed of task 103 (different shapes)",
  db.activities.find({"task_id": 103}, {"_id": 0}))
show("  Polymorphic query on a type-specific field",
  db.activities.find({"type": "file_upload"}, {"_id": 0, "file_name": 1, "size_kb": 1}))

# --- Bucket: group many small time-log entries into one document per user per day
for user_id, minutes in [(1, 45), (1, 30), (1, 60), (3, 50)]:
  db.time_buckets.update_one(
    {"user_id": user_id, "day": "2026-10-01", "count": {"$lt": 100}}, # max 100 entries per bucket
    {"$push": {"entries": {"minutes": minutes}},
     "$inc": {"count": 1, "total_minutes": minutes}},
    upsert=True,
  )
show("7. BUCKET pattern: one document per user per day",
  db.time_buckets.find({}, {"_id": 0}).sort("user_id", ASCENDING))

# --- Attribute: varying fields stored as key/value pairs so ONE index covers all of them
db.products.insert_many([
  {"name": "Task Hub Pro licence", "attrs": [{"k": "seats", "v": 25}, {"k": "billing", "v": "yearly"}]},
  {"name": "Task Hub Starter", "attrs": [{"k": "seats", "v": 5}, {"k": "trial_days", "v": 14}]},
])
db.products.create_index([("attrs.k", ASCENDING), ("attrs.v", ASCENDING)])
show("8. ATTRIBUTE pattern: plans with more than 10 seats",
  db.products.find({"attrs": {"$elemMatch": {"k": "seats", "v": {"$gt": 10}}}}, {"_id": 0, "name": 1}))

# --- Computed: keep a pre-calculated summary updated on every write (no re-aggregation on read)
db.projects.insert_one({"_id": "Backend", "task_count": 0, "total_hours": 0})
for task in db.tasks.find({"project": "Backend"}):
  db.projects.update_one({"_id": "Backend"}, {"$inc": {"task_count": 1, "total_hours": task["hours"]}})

```

```

show("9. COMPUTED pattern: pre-computed project totals", db.projects.find({"_id": "Backend"}))

# --- Outlier: a few documents are far larger than the rest; keep the first N inline,
#       flag the document and move the overflow to a separate collection
WATCHER_LIMIT = 3
watchers = [{"user_id": i} for i in range(1, 8)]          # 7 watchers, normal tasks have 1-2
db.tasks.update_one({"_id": 105}, {"$set": {"watchers": watchers[:WATCHER_LIMIT], "has_overflow": True}})
db.watchers_overflow.insert_one({"task_id": 105, "watchers": watchers[WATCHER_LIMIT:]})
show("10. OUTLIER pattern: inline watchers + overflow flag",
     db.tasks.find({"_id": 105}, {"_id": 1, "watchers": 1, "has_overflow": 1}))
show("    Overflow document", db.watchers_overflow.find({}, {"_id": 0}))
client.close()

```

Output

python aggregation_patterns.py

```

--- 1. $match + $group + $sort: hours per project (excluding TODO) ---
{'_id': 'Backend', 'tasks': 3, 'total_hours': 26, 'avg_hours': 8.666666666666666}
{'_id': 'Frontend', 'tasks': 1, 'total_hours': 9, 'avg_hours': 9.0}

--- 2. $project: computed fields ---
{'title': 'Design ER diagram', 'hours': 6, 'is_done': True, 'tag_count': 2, 'effort': 'small'}
{'title': 'Create PostgreSQL schema', 'hours': 8, 'is_done': True, 'tag_count': 2, 'effort': 'large'}
{'title': 'Write FastAPI CRUD', 'hours': 12, 'is_done': False, 'tag_count': 2, 'effort': 'large'}
{'title': 'Add JWT authentication', 'hours': 2, 'is_done': False, 'tag_count': 2, 'effort': 'small'}

--- 3. $unwind + $group + $sort + $limit: top 3 tags ---
{'_id': 'api', 'count': 4}
{'_id': 'db', 'count': 2}
{'_id': 'ui', 'count': 2}

--- 4. $lookup (JOIN tasks -> users) + $unwind + $project ---
{'title': 'Write FastAPI CRUD', 'assignee': 'Habeeb', 'team': 'Backend'}
{'title': 'Build dashboard page', 'assignee': 'Ananya Rao', 'team': 'Frontend'}

--- 5. Team workload report ($lookup + $group + $sort) ---
{'members': ['Habeeb', 'Rahul Verma'], 'open_tasks': 2, 'hours': 28, 'team': 'Backend'}
{'members': ['Ananya Rao', 'Sneha Reddy'], 'open_tasks': 2, 'hours': 10, 'team': 'Frontend'}

--- 6. POLYMORPHIC pattern: activity feed of task 103 (different shapes) ---
{'type': 'comment', 'task_id': 103, 'user_id': 1, 'text': 'Started GET /tasks'}
{'type': 'status_change', 'task_id': 103, 'user_id': 1, 'from': 'TODO', 'to': 'IN_PROGRESS'}

--- Polymorphic query on a type-specific field ---
{'file_name': 'er.png', 'size_kb': 292}

--- 7. BUCKET pattern: one document per user per day ---
{'user_id': 1,
 'day': '2026-10-01',
 'entries': [{'minutes': 45}, {'minutes': 30}, {'minutes': 60}],
 'count': 3,
 'total_minutes': 135}
{'user_id': 3, 'day': '2026-10-01', 'entries': [{'minutes': 50}], 'count': 1, 'total_minutes': 50}

--- 8. ATTRIBUTE pattern: plans with more than 10 seats ---
{'name': 'Task Hub Pro licence'}

--- 9. COMPUTED pattern: pre-computed project totals ---
{'_id': 'Backend', 'task_count': 4, 'total_hours': 28}

--- 10. OUTLIER pattern: inline watchers + overflow flag ---
{'_id': 105, 'watchers': ['user_1', 'user_2', 'user_3'], 'has_overflow': True}

--- Overflow document ---
{'task_id': 105, 'watchers': ['user_4', 'user_5', 'user_6', 'user_7']}

```

Result

The pipelines produced hours per project, computed fields, the top three tags, task-assignee joins through \$lookup and a team workload report. The Polymorphic pattern stored three activity shapes in one collection, the Bucket pattern grouped four time logs into two daily documents, the Attribute pattern answered a key/value query with one compound index, the Computed pattern kept project totals (4 tasks, 28 hours) up to date on write, and the Outlier pattern moved excess watchers into an overflow collection.